

Contents

1 - Mint (create the wallet) 2

2 - Spend (base M-of-N + delegated authorizations) 3

3 - Update config (per-UTxO) 4

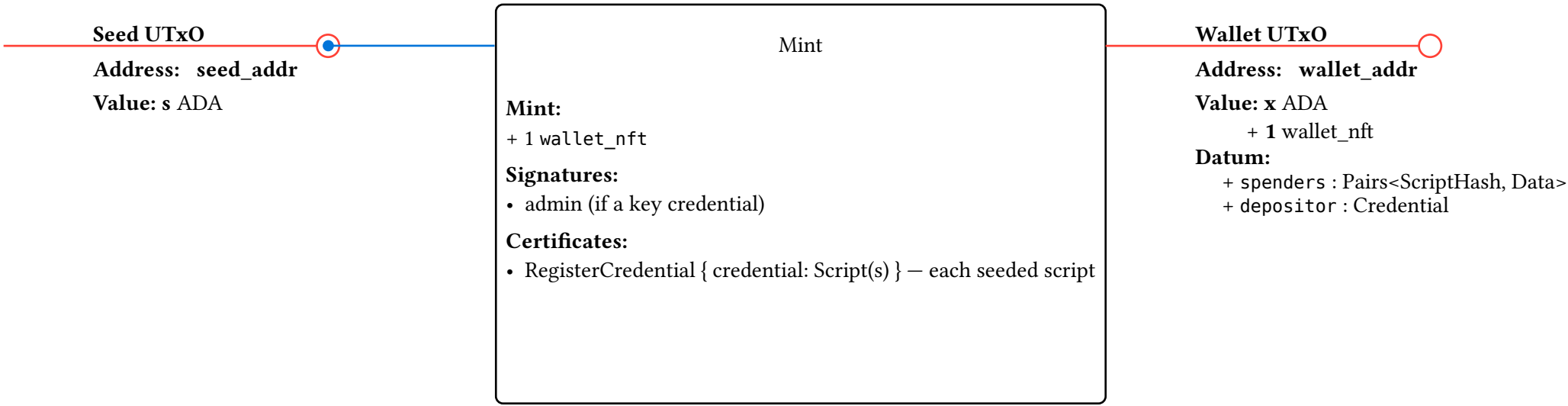
4 - Deposit (depositor adds funds) 5

5 - Close (admin burns the NFT) 6

6 - Example: stateful spending window (withdrawal script) 7

1 - Mint (create the wallet)

The wallet is created by minting its identifying NFT — one token under the script’s own policy — into a fresh wallet UTxO at the wallet address. The admin must authorize the mint.



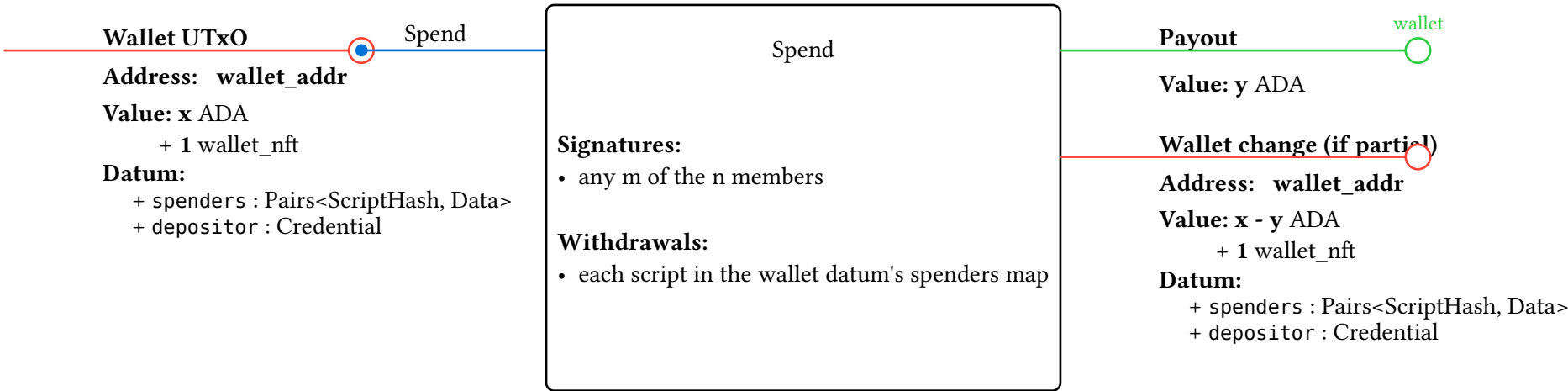
Notes:

- wallet_addr: payment is wallet_hash, staking any. wallet_hash = hash(smart_wallet(seed_utxo, settings_ref, wallet_nft_name, admin)) — the multivalidator is parameterized by its seed, configuration reference, NFT name, and the admin credential, so all wallet UTxOs share one address and one NFT policy.
- one-shot: the mint requires the seed_utxo to be spent, and a UTxO can be spent at most once, so this policy id mints exactly one token ever. The resulting NFT is unforgeable and identifies the wallet.
- the mint redeemer is Mint { out_ix }, locating the new wallet UTxO.
- the new wallet UTxO carries the NFT, an inline datum with a spenders map and a depositor credential, and no reference script. Every script in the spenders map must be registered in this same transaction — a RegisterCredential certificate for its stake credential, which runs the script’s publish handler. The handler reads its own initial Data from this output’s datum and validates it, so a wallet may be created already carrying delegated restrictions, but only ones that self-validated their initial state.
- the admin credential (a script parameter) must be satisfied: a key admin signs; a script admin authorizes by a withdraw-0 invocation.

Figure 1: Mint (create the wallet)

2 - Spend (base M-of-N + delegated authorizations)

The wallet pays out. The spending script enforces its base M-of-N signature floor and forwards every other check to the withdrawal scripts named in the wallet UTxO's datum; the single spend redeemer asserts their conjunction.

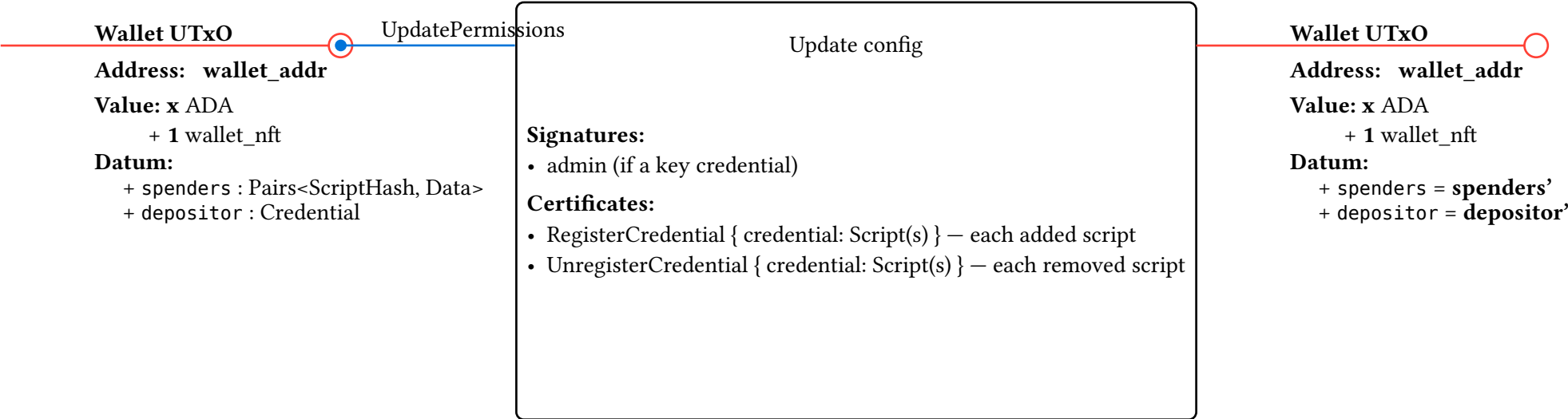


- Notes:**
- **wallet_addr:** payment is `wallet_hash`, staking any. A wallet UTxO holds its identifying NFT plus ada and arbitrary native assets.
 - `wallet_hash = hash(smart_wallet(seed_utxo, settings_ref, wallet_nft_name, admin))`: the spending script is parameterized by its seed, configuration reference, NFT name, and the `admin` credential, so all wallet UTxOs share one address. Externalizing the members/threshold config lets it change without redeploying the script.
 - **validity:** the spend requires the wallet UTxO to carry its NFT (1 `wallet_nft` under `wallet_hash`) — only NFT-bearing UTxOs are valid smart wallets.
 - **base M-of-N floor:** the wallet's configuration supplies members and threshold, and the spend requires at least threshold of members present in `extra_signatories`. A member may itself be a script credential, so the floor can be another multisig, a DAO, and so on. This design is agnostic of how that configuration is sourced.
 - **delegated authorizations:** every `ScriptHash` key in the wallet UTxO's `spenders` datum field is a `withdraw-0` staking script that must run in this transaction; the `Data` value carries that script's own mutable state. The Spend redeemer asserts the conjunction — the M-of-N floor **and** all delegated checks — so no restriction can be silently omitted. The map is per-UTxO, so different wallet UTxOs may carry different restriction sets.
 - the delegated scripts are the extension point: spending limits, time locks, recipient allow-lists, oracle attestations, ... each is an independent withdrawal script observed here, none known to the spending script.
 - **composability:** the spend asserts only its own inputs and that the required spenders ran — never the total input or output counts, nor unrelated UTxOs. Any partial change keeps the NFT so the wallet stays valid.

Figure 2: Spend (base M-of-N + delegated authorizations)

3 - Update config (per-UTxO)

An authorized admin credential — a parameter of the spending script — rewrites a wallet UTxO’s config (spenders and depositor) in place. Funds are untouched: same address, same value — only the config changes.



Notes:

- the admin credential (a script parameter) must be satisfied: a key admin signs; a script admin authorizes by a withdraw-0 invocation (the same pluggable mechanism used elsewhere).
- the continuation must sit at the same wallet_addr and carry the **same value** — including the NFT — so an update cannot move funds, only swap the config. The admin is fixed in the script parameter, so control cannot be handed off.
- spenders' / depositor': the new config. It takes effect immediately for subsequent spends of this UTxO; other wallet UTxOs are unaffected (the config is per-UTxO).
- the update diffs the old and new maps: every script **added** must be registered here (RegisterCredential, running its publish handler, which reads the script’s initial Data from the continuation datum and validates it); every script **removed** must be unregistered (UnregisterCredential, whose publish handler only accepts the unregister when it sees the script leaving the wallet); every script **kept** must carry unchanged Data. The depositor is freely rewritten by the admin. This ensures each entry’s Data was self-validated at registration and stays coherent across updates.

Figure 3: Update config (per-UTxO)

4 - Deposit (depositor adds funds)

The wallet’s `depositor` credential adds funds to the wallet UTxO — spending it and recreating it at the same address with the NFT, an unchanged datum, and no fund removed. It is the only non-admin, non-M-of-N way to touch the wallet, and it cannot spend funds or change the config.

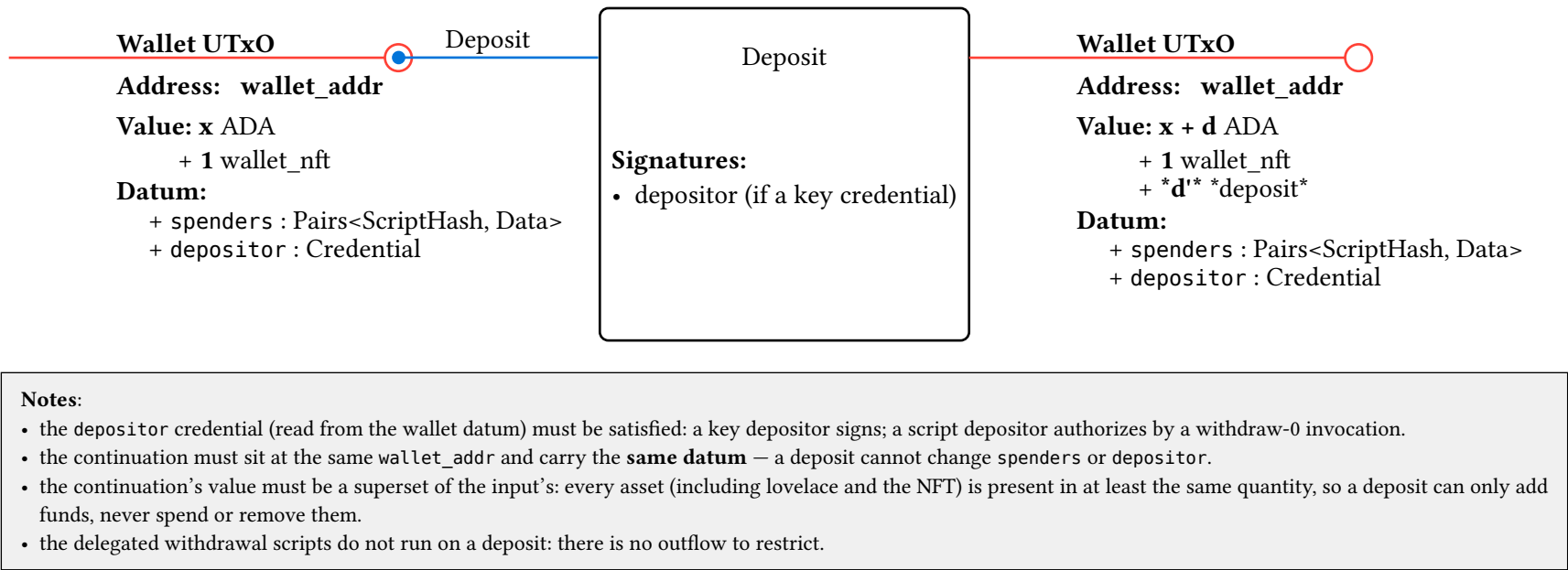
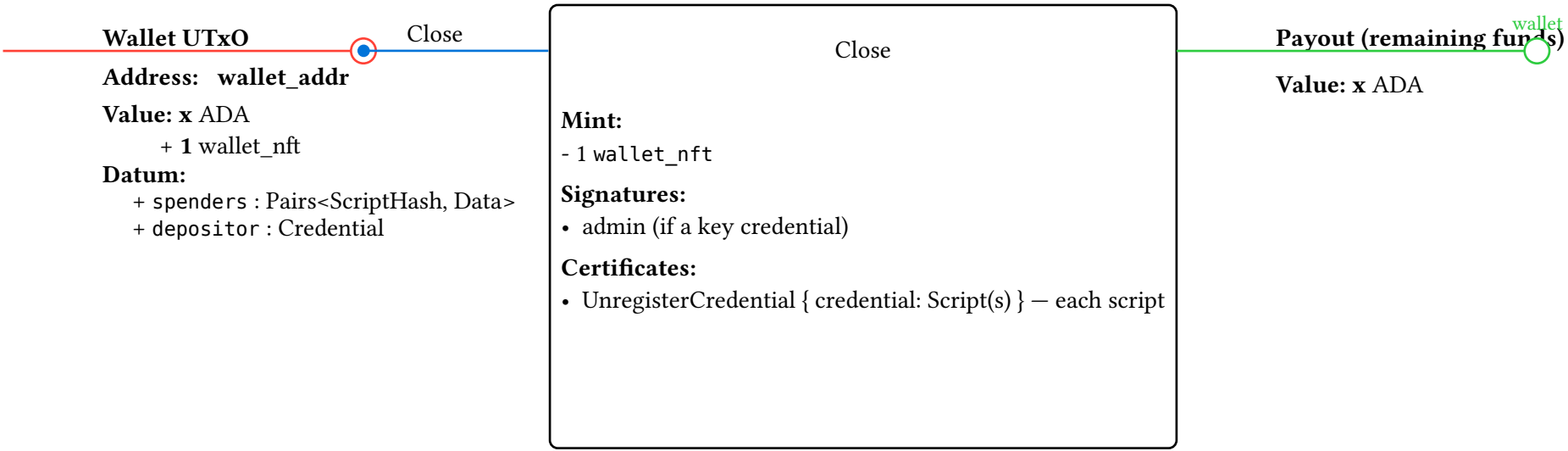


Figure 4: Deposit (depositor adds funds)

5 - Close (admin burns the NFT)

The admin closes the wallet: the wallet UTxO is spent, releasing whatever funds remain, and the identifying NFT is burned. The Close spend redeemer requires the admin, the burn, and that every delegated withdrawal script is unregistered so its stake deposit is refunded; the Burn mint redeemer permits the burn.



- Notes:**
- the admin credential (a script parameter) must be satisfied: a key admin signs; a script admin authorizes by a `withdraw-0` invocation.
 - the wallet NFT — 1 `wallet_nft` under `wallet_hash` — must be burned in this transaction (-1), destroying the wallet's identity. The burn runs the mint endpoint with a Burn redeemer, which only checks the burn quantity; authorization is enforced by the Close spend redeemer.
 - closing spends the wallet UTxO and releases any remaining funds. There is no continuation: once the NFT is gone, the wallet can no longer be spent.
 - every script in the closing wallet's `spenders` map must be unregistered here (`UnregisterCredential`), refunding its stake deposit. Each script's publish handler accepts this because it sees itself in the wallet input and no wallet output.

Figure 5: Close (admin burns the NFT)

6 - Example: stateful spending window (withdrawal script)

A stateful delegated withdrawal script that limits the wallet to at most one Spend per window. Unlike `spending_limit` (whose bound is fixed at compile time), this script carries mutable state — the last spend time — in the wallet datum’s `spenders` map, and enforces its own state transition on every spend.

The script is parameterized by `wallet` and `window` (milliseconds), and its state is a single timestamp: `SpendingWindowState { last_spend: Int }` (POSIX ms; 0 means never spent).

- **Publishing** (`RegisterCredential`): the publish handler reads its initial Data from the wallet output datum and requires `last_spend == 0` — an unspent wallet. Unregistering is only accepted while leaving the wallet (the same input/output presence check as `spending_limit`).
- **Withdraw** (runs on every `Spend`): the handler reads its current state from the wallet **input** datum and the next state from the wallet **output** datum, takes `now` from the validity range’s lower bound, and requires that the cooldown elapsed (`now - last_spend >= window`, or it is the first spend) and that the output state records exactly this spend (`next.last_spend == now`).
- Because the wallet’s `Spend` branch doesn’t constrain the continuation datum, the script enforces its own transition end-to-end: the builder must produce a continuation carrying the updated state, or the withdrawal fails and the transaction is rejected.

This is the template for any stateful restriction (spending budgets, time locks, allow-lists that mutate, ...): the state lives in `spenders[own_hash]`, the publish handler validates the initial value, and the `withdraw` handler reads the input state, validates the transition, and checks the output state.